PRODUCT CASE STUDY

Amazon Discovery Intelligence

A pipeline that reads a week of public Amazon reviews and names the one problem worth starting. For every other problem it says what evidence is missing.

Ritika Das · ritikadas.in · August 2026

THE PROBLEM

The reading is done. The deciding is not.

A product manager owns one surface and gets a few hundred complaints a week across three stores. She has one afternoon. So she skims, and skimming never tells you what you missed.

What happens now	Sort by star rating, read the loudest, pick one, build it.
What that misses	The complaint that cost somebody money was short and calm. It never got picked.
What is actually hard	Not reading the reviews. Choosing which single problem to start on Monday, and being able to defend that choice.

PRIMARY RESEARCH

Four products already claim this job

I went through what each one takes in and what it hands back. Reading and grouping is solved. All four stop at the same line, one step before the decision.

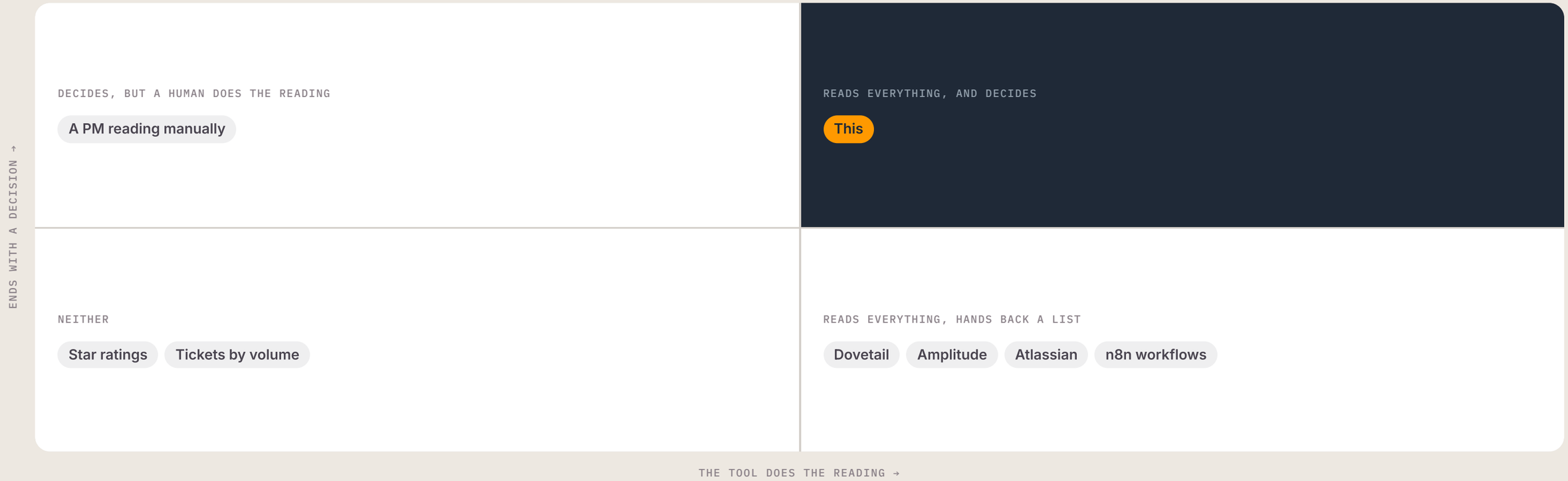
PRODUCT	WHAT IT DOES	WHERE IT STOPS
Dovetail	A store for customer feedback. Reads raw text and groups it into themes. Sold to Canva, Meta and NTT DOCOMO.	A list
Amplitude	Product analytics that added feedback clustering, so themes sit beside behaviour data.	A list
Atlassian	An agent that reads the feedback queue for its own PMs. Atlassian publishes a saving of about 40 minutes a day per PM.	A list
An n8n workflow	A no-code visual workflow that pulls reviews and summarises them. Free and quick, and hard to change once the logic gets real.	A summary

Product surfaces and published vendor material. None of the four names a first move, an owner or a cost, and none of them refuses to rank a theme it cannot back. The 40-minute figure is Atlassian's own published claim, not a measurement of mine.

COMPETITIVE LANDSCAPE

The gap is one step wide

Two questions separate the field. Does the tool do the reading for you? And does it end with a decision, or with a list?



PERSONA AND JOBS TO BE DONE

One user, and three jobs she cannot finish

PRIMARY USER

PM on a consumer app

Owns one surface, such as Checkout or Search. A few hundred public complaints a week across three stores, and one afternoon. Reports to a lead who asks why this and not that.

When a week of complaints lands, **I want to know which single problem to start on**, so I can spend the afternoon building instead of reading.

When I pick a problem, **I want the evidence underneath it in one place**, so I can defend the choice to my lead without opening the reviews again.

When the evidence is thin, **I want to be told that plainly**, so I do not confidently build the wrong thing.

PROBLEM PRIORITISATION

What I chose to build, and what I dropped

Five candidates. I scored each on whether it closed a job above, and on whether anyone had already closed it.

CANDIDATE	REASONING	CALL
Group reviews better	The four existing products already do this well. Rebuilding it adds nothing.	Dropped
Name the first move	The step every one of them stops before. Closes job one and job two together.	Built
Refuse to rank thin evidence	Closes job three. Nothing on the market does it, and it is what stops confident nonsense.	Built
Chat over the review corpus	Answers questions about the week and cites the reviews behind each answer. Support for the decision, not the decision itself.	Built
Week-by-week history view	Nobody asks what happened in week 22. They ask what is going wrong in Checkout now.	Dropped

MVP SCOPE

The smallest thing that tests the claim

The claim is that a PM can act on the output without opening a single review. Anything that did not test that claim waited.

In scope

- \+ Three live public sources: App Store, Play Store, Amazon product pages
- \+ A substance filter that drops “good app” and emoji-only, in any language
- \+ Grouping into named problems by a language model
- \+ Every count done in code: people, sources, score, change on last week
- \+ RICE and MoSCoW ranking, plus a second score for what the problem cost
- \+ A readiness gate that refuses to rank what it cannot back
- \+ One first move, an owner and a rough cost on every problem
- \+ A weekly run, a Google Sheet, an email digest and a dashboard
- \+ A chat over the week that cites the reviews behind every answer

Out of scope, on purpose

- − Sign-in. Every endpoint is open today, and that is a known gap
- − A paid reviews API, which would unblock iOS and critical reviews
- − Reddit as a fourth source
- − Vector search under the chat
- − A browsable archive of past weeks
- − Writing the chosen move back into Jira
- − More than one product at a time

THE SOLUTION

Fourteen steps. The model is called at four of them.

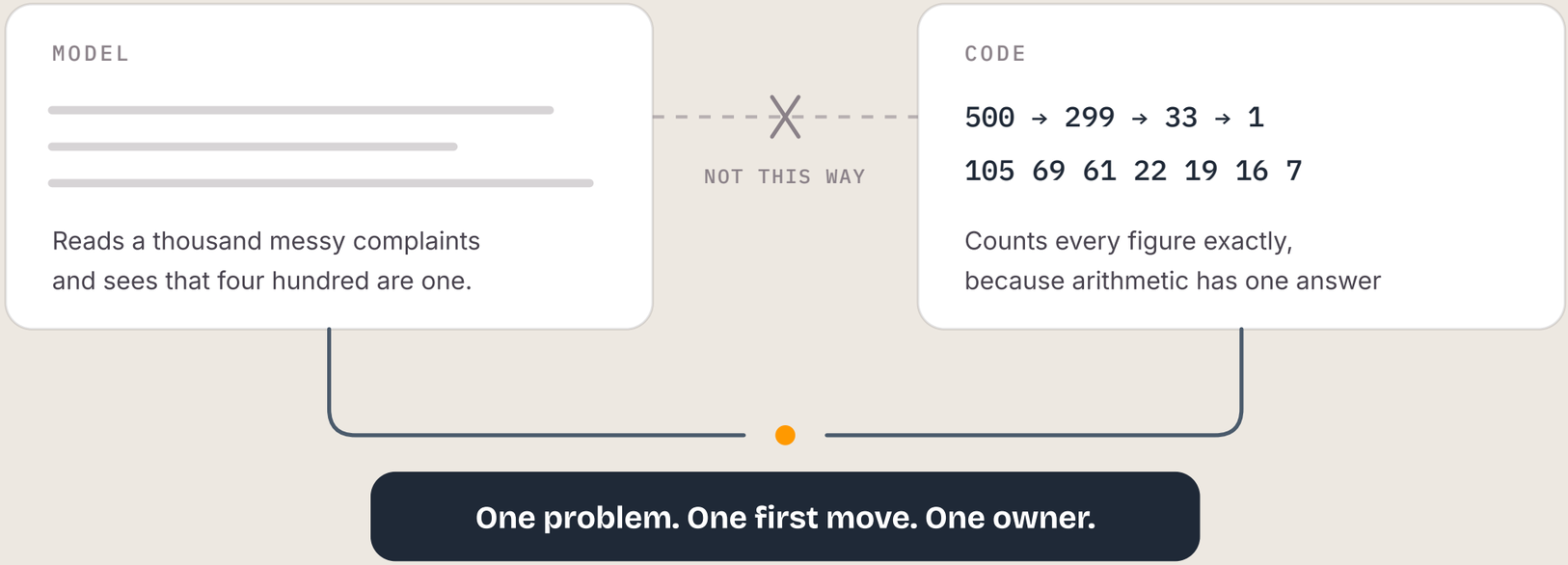
None of the four model calls does arithmetic. Every number a PM reads is counted in code, where it cannot drift.

01	**Collect**	App Store, Play Store and Amazon product pages, once a week
02	**Filter**	A substance bar drops praise, noise and emoji-only, in any language
03	**Group**	The model reads and clusters complaints into named problems
04	**Count**	Code does every figure: people, sources, score, change on last week
05	**Gate**	Refuses to rank a problem it cannot back, and names what is missing
06	**Deliver**	A digest that leads with one first move, an owner and a price

THE RULE UNDERNEATH

Never ask a model to grade what you have already counted

The model is not the weak half. It reads a thousand badly written complaints and sees that four hundred are one problem, which is something I cannot do. Code does the arithmetic, which has exactly one right answer. Give each of them the job it is good at, and together they do more than either does alone.



I learned the rule the expensive way. I asked the model to grade a count code had already made exactly. It looked at 53 complaints and wrote “only one person reported this”, and the page rendered that beside the number 53.

ONE LIVE WEEK

What a week produces, and where it lands

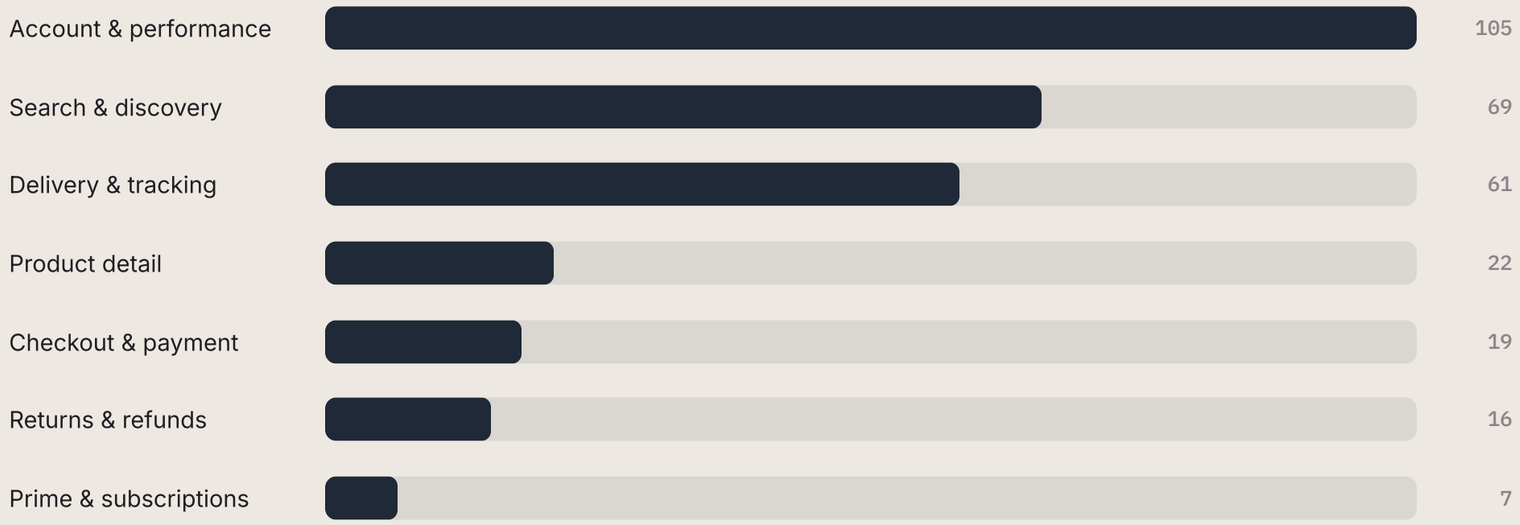
The last number in the funnel is the whole product. The other four tools would have ranked all 33 and let the PM find out for herself which ones were thin.

500 COLLECTED
From three sources

299 SORTED
Enough substance to place

33 PROBLEMS
Across 7 parts of the app

1 READY
Enough evidence to act on



Live run, 18 August 2026, and the figures move every week. The bars are the 299 sorted complaints and sum to 299. By source: App Store 291, Play Store 196, Amazon product pages 13. Apple blocks requests from datacentre addresses, so App Store volume depends on where the run happens — see v2.

THE FINDING THAT CHANGED THE PRODUCT

Severity measures loudness, not cost

Someone who cannot find a product writes a furious review. Someone charged twice writes a short, tired one. Ranking on tone alone buries the money.

RANKED ON TONE ALONE

3.2

The upset score on the week’s costliest problem. Unremarkable. The 23 complaints that cost people money sit across 7 problems, most of them below problems that only irritated people.

TWO SCORES, NOT ONE

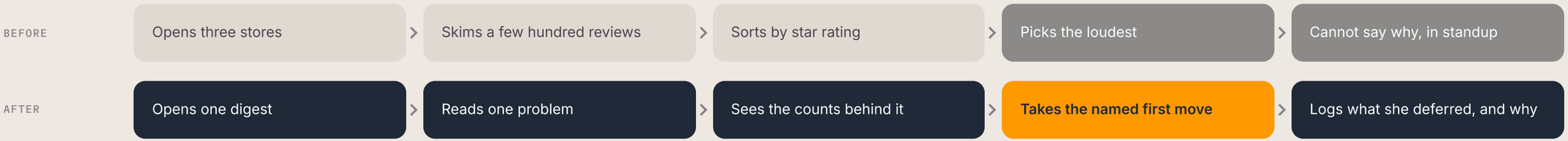
44

People on the problem that came top. 13 packages lost or sent to the wrong address, 2 charged for items that never arrived. The two scores disagree often enough to earn the second look.

USER JOURNEY

One Monday afternoon, before and after

It does not save her the reading. It saves her the deciding, which is the part she could not defend.



BUSINESS CASE

What it costs, what it saves, and how I would know

An internal tool, so the case is cost against time, not revenue. The last two lines are the ones an interviewer should press on.

LINE	DETAIL	FIGURE
Cost to run	Cloud Run scales to zero between weekly runs. It runs once a week whoever reads it, so adding readers costs nothing.	₹40/mo
The alternative	A managed Postgres database to hold seven rows a week, which is what the first design used.	₹1,100/mo
The value to beat	Atlassian’s published saving for its own PMs, from an agent that reads the queue and still hands back a list.	40 min/day
How I would know	Decision rate: the share of surfaced problems a PM acts on or deliberately defers. Beside it, how often the product calls a problem ready when the evidence cannot carry it.	The metric
What is not proven	No PM has run a live week through this one yet, so the saving above is a comparable product’s published figure, not a result of mine.	–

TRADE-OFFS

Three calls, and what each one bought

A spreadsheet over a database

Gave up queries and schema control. Got a store PMs already sort, filter and share, and ₹1,100 a month became ₹40.

Their schema over mine

Gave up a clean schema of my own. Kept every filter and pivot a PM had already built on the sheet, misspelled column header and all.

The question over the archive

Gave up the week-by-week history view. Nobody asks what happened in week 22. They ask what is going wrong in Checkout.

V2 PLANNING

What ships next, in order

Ordered by what unblocks the most, not by what is most interesting to build.

NEXT	WHY IT IS NEXT	UNBLOCKS
A paid reviews API	Apple blocks datacentre addresses, so a scheduled run can come back with nothing from iOS. Critical reviews are also behind a sign-in.	Volume
Sign-in	Every endpoint is publicly callable today. This has to close before anyone else uses it.	Real users
Reddit as a fourth source	Platform complaints live there and never reach a store review.	Coverage
Vector search under the chat	The chat is built and holds one week inside a single prompt. Vector search is what it needs once the corpus outgrows that.	Depth
Track whether the move gets taken	The digest already carries a feedback link. This turns the decision rate from a plan into a number.	The proof

LIVE NOW

amazon.ritikadas.in

Runs every Monday at 09:00, scales to zero between runs, about ₹40 a month. The full build write-up is at ritikadas.in/work/amazon.